Name	AYUSHI JAPSARE
UID no.	2023301006
EXP	2

CSS	
AIM:	To implement: i) Rail Fence Cipher for both encryption and decryption of a message. ii) Columnar Transposition Cipher for both encryption and decryption of a message. iii) Double Transposition Cipher for both encryption and decryption of a message.
PROBLEM STATEMENT:	Problem Definition – Transposition ciphers rearrange the letters of plaintext without altering the actual characters. This results in ciphertext where the letter frequency remains unchanged but the order is scrambled. In this experiment, we focus on three transposition ciphers: 1. Rail Fence Cipher – Characters are written in a zig-zag pattern across rails and read row by row. 2. Columnar Transposition Cipher – Plaintext is arranged in a matrix with columns defined by a keyword, then read column by column in a permuted order. 3. Double Transposition Cipher – Applies columnar transposition twice with two different keys, making it more secure. These techniques illustrate the concept of permutation, key-based reordering, and provide insight into classical cryptography's transposition approach.
THEORY	**Understanding and Learning Summary** This experiment provided comprehensive hands-on experience with three classical transposition cipher algorithms: Rail Fence Cipher, Columnar Transposition Cipher, and Double Transposition Cipher. Unlike substitution ciphers, these algorithms rearrange the positions of characters without changing the characters themselves, demonstrating the fundamental cryptographic principle of confusion through permutation.

	Key Concepts Learned 1. Rail Fence Cipher The Rail Fence cipher arranges plaintext in a zigzag pattern across multiple rails and reads it row-wise. Key learnings include: ● Zigzag Pattern Implementation: Understanding directional changes at rail boundaries ● Matrix Representation: Using 2D arrays to visualize character placement ● Pattern Recognition: The cipher creates predictable patterns that can be exploited ● Simplicity vs Security: While conceptually simple, it offers limited security due to pattern predictability 2. Columnar Transposition Cipher This cipher arranges plaintext in a rectangular matrix and reads columns in a key-determined order. Key insights include: ● Key-Based Permutation: Converting alphabetic keys into numerical column orders ● Matrix Operations: Efficient text arrangement and retrieval using row-column indexing ● Padding Mechanism: Handling text lengths that don't perfectly fit the matrix ● Alphabetical Ordering: Implementing stable sorting for consistent column ordering with repeated letters 3. Double Transposition Cipher Applying columnar transposition twice with different keys significantly enhances security. Learning outcomes include: ● Cascading Operations: Understanding how multiple cipher applications compound security ● Intermediate Processing: Managing data flow between cipher stages ● Security Enhancement: Demonstrating how complexity increases exponentially with multiple keys ● Reversibility: Ensuring proper decryption by applying operations in reverse order

CODE:

```
import math

class RailFenceCipher:
    """Implementation of Rail Fence Cipher with encryption and decryption"""

    @staticmethod
    def encrypt(plaintext, rails):
        """Encrypt plaintext using Rail Fence cipher"""
        if rails == 1:
            return plaintext

        # Remove spaces and convert to uppercase
        text = plaintext.replace(' ', '').upper()

        # Create rail matrix
        rail_matrix = [['\n' for _ in range(len(text))] for _ in range(rails)]

        # Direction flag: True for down, False for up
        direction_down = False
        row = 0

        # Fill the rail matrix in zigzag pattern
        for col in range(len(text)):
            # Check direction change
            if row == 0 or row == rails - 1:
                direction_down = not direction_down

            # Place character
            rail_matrix[row][col] = text[col]

            # Move to next row
            if direction_down:
                row += 1
            else:
                row -= 1

        # Read matrix row by row to create ciphertext
        ciphertext = ""
        for i in range(rails):
            for j in range(len(text)):
                if rail_matrix[i][j] != '\n':
                    ciphertext += rail_matrix[i][j]
```

```
        return ciphertext

    @staticmethod
    def decrypt(ciphertext, rails):
        """Decrypt ciphertext using Rail Fence cipher"""
        if rails == 1:
            return ciphertext

        # Create rail matrix
        rail_matrix = [['\n' for _ in range(len(ciphertext))] for _ in range(rails)]

        # Mark the positions in zigzag pattern
        direction_down = None
        row = 0

        for col in range(len(ciphertext)):
            if row == 0:
                direction_down = True
            if row == rails - 1:
                direction_down = False

            # Mark this position
            rail_matrix[row][col] = '*'

            # Move to next row
            if direction_down:
                row += 1
            else:
                row -= 1

        # Fill the marked positions with ciphertext characters
        index = 0
        for i in range(rails):
            for j in range(len(ciphertext)):
                if rail_matrix[i][j] == '*' and index < len(ciphertext):
                    rail_matrix[i][j] = ciphertext[index]
                    index += 1

        # Read the matrix in zigzag pattern to get plaintext
        plaintext = ""
        direction_down = None
        row = 0

        for col in range(len(ciphertext)):
```

```
            if row == 0:
                direction_down = True
            if row == rails - 1:
                direction_down = False

            # Add character to plaintext
            if rail_matrix[row][col] != '*':
                plaintext += rail_matrix[row][col]

            # Move to next row
            if direction_down:
                row += 1
            else:
                row -= 1

        return plaintext

class ColumnarTranspositionCipher:
    """Implementation of Columnar Transposition Cipher with encryption and
decryption"""

    def __init__(self, key):
        self.key = key.upper()
        self.key_order = self._get_key_order()

    def _get_key_order(self):
        """Get the column order based on alphabetical sorting of key"""
        # Create list of (character, original_index) pairs
        indexed_key = [(char, i) for i, char in enumerate(self.key)]
        # Sort by character, then by original index for stability
        sorted_key = sorted(indexed_key)
        # Return the order mapping
        order = [0] * len(self.key)
        for new_pos, (char, old_pos) in enumerate(sorted_key):
            order[old_pos] = new_pos
        return order

    def encrypt(self, plaintext):
        """Encrypt plaintext using Columnar Transposition cipher"""
        # Remove spaces and convert to uppercase
        text = plaintext.replace(' ', '').upper()

        # Calculate number of rows needed
        cols = len(self.key)
```

```
    rows = math.ceil(len(text) / cols)

    # Pad text if necessary
    padded_text = text + 'X' * (rows * cols - len(text))

    # Create matrix
    matrix = []
    for i in range(rows):
        row = []
        for j in range(cols):
            if i * cols + j < len(padded_text):
                row.append(padded_text[i * cols + j])
            else:
                row.append('X')
        matrix.append(row)

    # Read columns according to key order
    ciphertext = ""
    for order_pos in range(cols):
        # Find which original column has this order position
        col_index = self.key_order.index(order_pos)
        for row in range(rows):
            ciphertext += matrix[row][col_index]

    return ciphertext

def decrypt(self, ciphertext):
    """Decrypt ciphertext using Columnar Transposition cipher"""
    cols = len(self.key)
    rows = len(ciphertext) // cols

    # Create empty matrix
    matrix = [['' for _ in range(cols)] for _ in range(rows)]

    # Fill matrix column by column according to key order
    index = 0
    for order_pos in range(cols):
        # Find which original column has this order position
        col_index = self.key_order.index(order_pos)
        for row in range(rows):
            if index < len(ciphertext):
                matrix[row][col_index] = ciphertext[index]
                index += 1
```

```
        # Read matrix row by row
        plaintext = ""
        for row in range(rows):
            for col in range(cols):
                plaintext += matrix[row][col]

        # Remove padding
        return plaintext.rstrip('X')

class DoubleTranspositionCipher:
    """Implementation of Double Transposition Cipher with encryption and
decryption"""

    def __init__(self, key1, key2):
        self.cipher1 = ColumnarTranspositionCipher(key1)
        self.cipher2 = ColumnarTranspositionCipher(key2)
        self.key1 = key1
        self.key2 = key2

    def encrypt(self, plaintext):
        """Encrypt plaintext using Double Transposition cipher"""
        # Apply first transposition
        intermediate = self.cipher1.encrypt(plaintext)
        # Apply second transposition
        ciphertext = self.cipher2.encrypt(intermediate)
        return ciphertext

    def decrypt(self, ciphertext):
        """Decrypt ciphertext using Double Transposition cipher"""
        # Apply second transposition decryption (reverse order)
        intermediate = self.cipher2.decrypt(ciphertext)
        # Apply first transposition decryption
        plaintext = self.cipher1.decrypt(intermediate)
        return plaintext

def display_menu():
    """Display the main menu"""
    print("\n" + "="*65)
    print("       CRYPTOGRAPHY LAB - TRANSPOSITION CIPHER
IMPLEMENTATION")
    print("="*65)
    print("1. Rail Fence Cipher")
    print("2. Columnar Transposition Cipher")
    print("3. Double Transposition Cipher")
```

```
    print("4. Quit")
    print("="*65)

def rail_fence_menu():
    """Handle Rail Fence cipher operations"""
    print("\n--- Rail Fence Cipher ---")
    try:
        rails = int(input("Enter number of rails (2 or more): "))
        if rails < 2:
            print("Number of rails must be 2 or more")
            return

        plaintext = input("Enter plaintext: ")

        # Encryption
        ciphertext = RailFenceCipher.encrypt(plaintext, rails)
        print(f"\nOriginal Text: {plaintext}")
        print(f"Number of Rails: {rails}")
        print(f"Encrypted Text: {ciphertext}")

        # Decryption
        decrypted = RailFenceCipher.decrypt(ciphertext, rails)
        print(f"Decrypted Text: {decrypted}")

        # Show rail pattern for visualization
        print(f"\nRail Pattern Visualization:")
        show_rail_pattern(plaintext.replace(' ', '').upper(), rails)

    except ValueError:
        print("Invalid input! Please enter a valid number for rails.")

def show_rail_pattern(text, rails):
    """Display the rail fence pattern for visualization"""
    if rails == 1:
        print(text)
        return

    # Create rail matrix for visualization
    rail_matrix = [[' ' for _ in range(len(text))] for _ in range(rails)]

    direction_down = False
    row = 0

    for col in range(len(text)):
```

```
        if row == 0 or row == rails - 1:
            direction_down = not direction_down

        rail_matrix[row][col] = text[col]

        if direction_down:
            row += 1
        else:
            row -= 1

    # Print the pattern
    for i in range(rails):
        print(f"Rail {i+1}: {''.join(rail_matrix[i])}")

def columnar_transposition_menu():
    """Handle Columnar Transposition cipher operations"""
    print("\n--- Columnar Transposition Cipher ---")
    key = input("Enter keyword: ").strip()

    if not key or not key.isalpha():
        print("Invalid key! Key must contain only alphabetic characters.")
        return

    plaintext = input("Enter plaintext: ")

    try:
        cipher = ColumnarTranspositionCipher(key)

        # Encryption
        ciphertext = cipher.encrypt(plaintext)
        print(f"\nOriginal Text: {plaintext}")
        print(f"Key: {key.upper()}")
        print(f"Column Order: {[i+1 for i in cipher.key_order]}")
        print(f"Encrypted Text: {ciphertext}")

        # Decryption
        decrypted = cipher.decrypt(ciphertext)
        print(f"Decrypted Text: {decrypted}")

    except Exception as e:
        print(f"Error: {e}")

def double_transposition_menu():
    """Handle Double Transposition cipher operations"""
```

```
    print("\n--- Double Transposition Cipher ---")
    key1 = input("Enter first keyword: ").strip()
    key2 = input("Enter second keyword: ").strip()

    if not key1 or not key1.isalpha() or not key2 or not key2.isalpha():
        print("Invalid keys! Keys must contain only alphabetic characters.")
        return

    plaintext = input("Enter plaintext: ")

    try:
        cipher = DoubleTranspositionCipher(key1, key2)

        # Encryption
        ciphertext = cipher.encrypt(plaintext)
        print(f"\nOriginal Text: {plaintext}")
        print(f"First Key: {key1.upper()}")
        print(f"Second Key: {key2.upper()}")
        print(f"Encrypted Text: {ciphertext}")

        # Decryption
        decrypted = cipher.decrypt(ciphertext)
        print(f"Decrypted Text: {decrypted}")

    except Exception as e:
        print(f"Error: {e}")

def main():
    """Main program loop"""
    print("Welcome to Cryptography Lab - Experiment 02!")
    print("Transposition Cipher Implementation")

    while True:
        display_menu()
        try:
            choice = input("Enter your choice (1-4): ").strip()

            if choice == '1':
                rail_fence_menu()
            elif choice == '2':
                columnar_transposition_menu()
            elif choice == '3':
                double_transposition_menu()
            elif choice == '4':
```

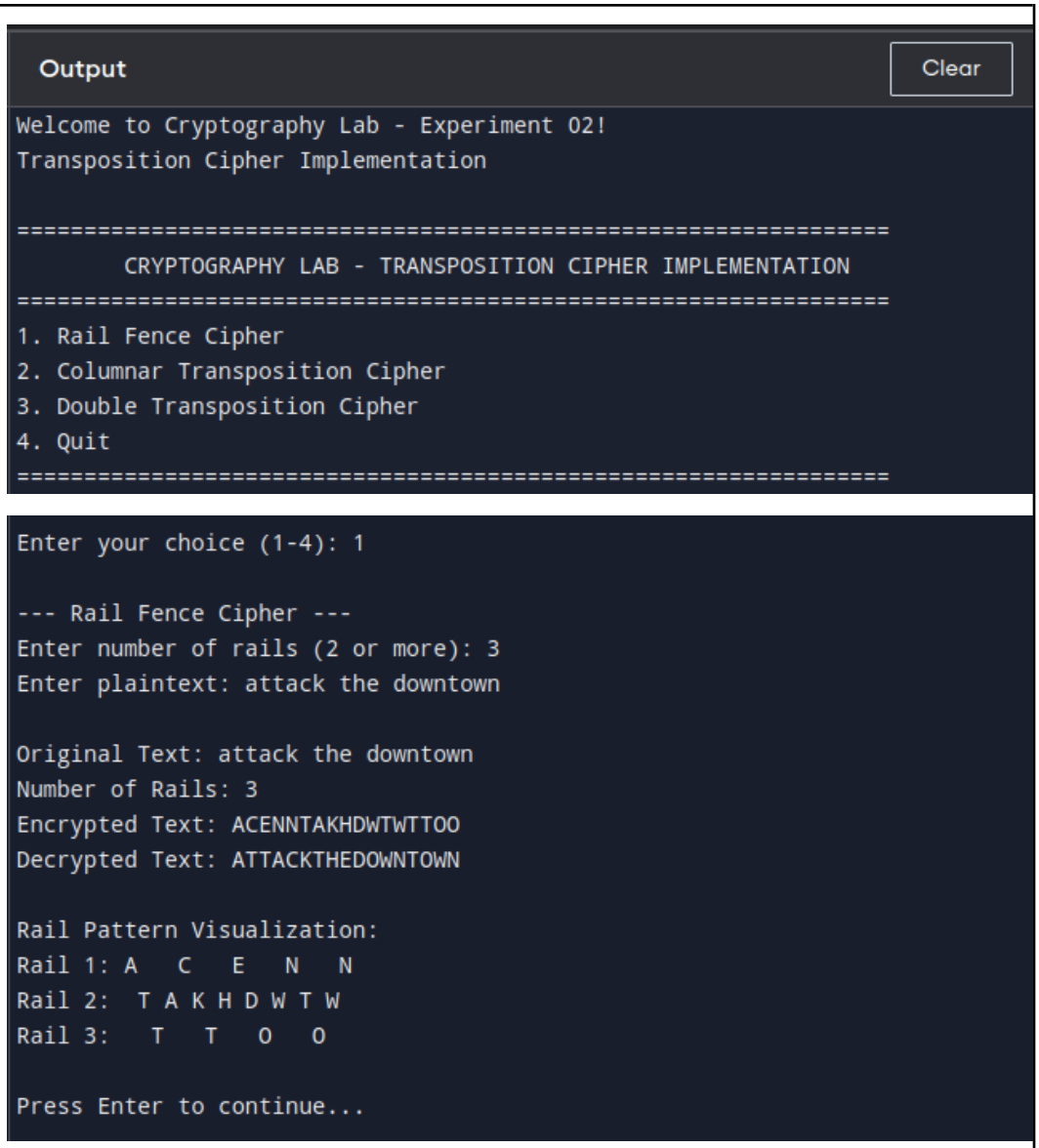
```
            print("\nThank you for using Cryptography Lab!")
            print("Goodbye!")
            break
        else:
            print("Invalid choice! Please enter 1, 2, 3, or 4.")

    except KeyboardInterrupt:
        print("\n\nProgram interrupted. Goodbye!")
        break
    except Exception as e:
        print(f"An error occurred: {e}")

    input("\nPress Enter to continue...")

if __name__ == "__main__":
    main()
```

OUTPUT:


Output
Clear
Welcome to Cryptography Lab - Experiment 02!
Transposition Cipher Implementation
CRYPTOGRAPHY LAB - TRANSPOSITION CIPHER IMPLEMENTATION
1. Rail Fence Cipher
2. Columnar Transposition Cipher
3. Double Transposition Cipher
4. Quit
Enter your choice (1-4): 1
--- Rail Fence Cipher ---
Enter number of rails (2 or more): 3
Enter plaintext: attack the downtown
Original Text: attack the downtown
Number of Rails: 3
Encrypted Text: ACENNTAKHDWTWTTOO
Decrypted Text: ATTACKTHEDOWNTOWN
Rail Pattern Visualization:
Rail 1: A C E N N
Rail 2: T A K H D W T W
Rail 3: T T O O
Press Enter to continue...

```
--- Rail Fence Cipher ---
Enter number of rails (2 or more): 5
Enter plaintext: lassi is safecode for entry

Original Text: lassi is safecode for entry
Number of Rails: 5
Encrypted Text: LAOASFFRSSEEEYSICDNRIOT
Decrypted Text: LASSIISSAFECODEFORENTRY

Rail Pattern Visualization:
Rail 1: L       A       O
Rail 2:  A     S F     F R
Rail 3:   S   S   E   E   E   Y
Rail 4:    S I     C D     N R
Rail 5:     I       O       T

Press Enter to continue...
```

```
Enter your choice (1-4): 2

--- Columnar Transposition Cipher ---
Enter keyword: donkey
Enter plaintext: lassi is safecode for entry

Original Text: lassi is safecode for entry
Key: DONKEY
Column Order: [1, 5, 4, 3, 2, 6]
Encrypted Text: LSOEIEOYSFFRSAETASDNICRX
Decrypted Text: LASSIISSAFECODEFORENTRY

Press Enter to continue...
=== Session Ended. Please Run the code again ===
```

```
================================================================
Enter your choice (1-4): 2

--- Columnar Transposition Cipher ---
Enter keyword: cargo
Enter plaintext: attack the downtown

Original Text: attack the downtown
Key: CARGO
Column Order: [2, 1, 5, 3, 4]
Encrypted Text: TTWNAKOWAETXCDOXTHNX
Decrypted Text: ATTACKTHEDOWNTOWN

Press Enter to continue...
```

	`` Enter your choice (1-4): 3 --- Double Transposition Cipher --- Enter first keyword: donkey Enter second keyword: happiness Enter plaintext: lassi is safecode entry Original Text: lassi is safecode entry First Key: DONKEY Second Key: HAPPINESS Encrypted Text: SEYNXXLFDIATEEXOXIRSCXAXSSX Decrypted Text: LSFNRXSAEXIAOEXSES Press Enter to continue... ``` ``` Enter your choice (1-4): 3 --- Double Transposition Cipher --- Enter first keyword: cargo Enter second keyword: attack Enter plaintext: attack the downtown Original Text: attack the downtown First Key: CARGO Second Key: ATTACK Encrypted Text: TOCNNEXXATTXKXHXTWDXWAOX Decrypted Text: NTCOEATDWTKWOA Press Enter to continue... ```
Conclusion:	The implementation successfully demonstrated the evolution of transposition-based cryptography from simple geometric patterns to complex multi-stage permutations. Understanding these classical techniques provides essential foundation knowledge for modern cryptographic systems, illustrating concepts like permutation matrices, key-based transformations, and the principle that security often emerges from mathematical complexity rather than algorithm secrecy. The hands-on programming experience reinforced theoretical understanding while developing practical skills in cryptographic algorithm implementation and analysis.